

PotensicProxy_TAF

PStart-Sessionprotokoll und CI-Runtime-Bereinigung

Implementierung v0.980 / versionCode 53

PStart v0.978 unverändert uebernommen; CI-Build gegen alte Pairing-Runtime-Quellen abgesichert und Frontend/Backend/Runtime auf v0.980 synchronisiert

Projektname	PotensicProxy_TAF / Potensic_App
Dokumentenart	Projektdokumentation / Implementierungsbericht
Erstellungsdatum	2026-10-11
Aenderungsdatum	2026-10-11
Verantwortlich	OpenAI ChatGPT - technische Ausarbeitung im Projektkontext
Version	v0.980 / Android versionCode 53
Status	Implementiert; CI-Runtime-Bereinigung technisch validiert, Flugtest der PStart-v0.978-Logik weiterhin ausstehend
Ablageort	PotensicProxy_TAF/DOCUMENTATION sowie Auslieferungspaket
Referenzen	PStart v0.978; Embedded-Frontend-Synchronisierung v0.979; CI-Log vom 2026-10-11; Regeln_Projektdokumentation_PDF_DOCX_Pflicht_.pdf

Ziel: Der PStart-Aufstieg soll insbesondere bei Wind und bei der ersten Luftreferenz robuster werden. Gleichzeitig muss jeder Lauf nachtraeglich zeitlich exakt rekonstruierbar sein - einschliesslich der von PStart gesendeten Steuerbefehle.

1. Aenderungshistorie

Version	Datum	Status	Thema	Aenderung
0.976	2026-10-08	Implementiert	PStart v1	Erste PStart-Automatik mit STEP0, Referenzstufen und EXIF/XMP.
0.977	2026-10-08	Buildfix	EXIF GPS	setLatLong-Buildfehler beseitigt; GPS-Tags explizit geschrieben.
0.978	2026-10-10	Implementiert	Aufstieg + Protokoll	Robustere Hoehenstufen, Positionstoleranz, Session-JSONL, Masterbild-Einbettung.
0.979	2026-10-10	Build-/Releasefix	Embedded Frontend	WebUI-Assets auf aktuellen Quellstand synchronisiert; Frontend/Backend/Runtime-Version pruefbar gekoppelt.
0.980	2026-10-11	Buildfix	CI Pairing Runtime	Alte taf-pairing-runtime-Dateien werden vor jedem WebUI-Build versionsgesteuert entfernt; Verifier prueft Source und Embedded-Ausgabe.

2. Ausgangsproblem

- Der erste Luftreferenzpunkt liegt weiterhin bei gemittelter stabiler Schwebelage + 0,10 m. Dieser kleine Höhenhub kann bei Wind, quantisierter Höhentelemetrie oder einem zu kleinen Throttle-Befehl schwer eindeutig erreicht werden.
- Die horizontale GPS-Position darf in PStart v1 nicht auf Zentimeter genau erzwungen werden, weil v1 absichtlich keine automatische Pitch-/Roll-/Yaw-Korrektur durchführt.
- Das bisherige Live-System-Log dokumentiert Ereignisse, ist aber kein dauerhaftes, sessionspezifisches Flugprotokoll. Für die Analyse muss sichtbar sein, was PStart befohlen hat und wie die Telemetrie darauf reagierte.
- Die STEP0-Aufnahmen bleiben Dokumentationsbilder und werden nicht als optische PLanding-Referenz verwendet.

3. Neuer Zielablauf

1. Normalen Start ausführen und einen stabilen Schwebestand erkennen.
2. Schwebelage über das stabile Zeitfenster mitteln; TOF wird weiterhin nicht für die PStart-Höhenregelung verwendet.
3. Ersten Sollpunkt als gemittelte Schwebelage + 0,10 m bilden.
4. Nur vertikal steigen; bei Sollwertübergang Throttle sofort neutralisieren.
5. Nicht auf einen einzelnen exakten Dezimeterwert bestehen, sondern die Vertikalbewegung tolerant beruhigen.
6. Gimbal -90 Grad bestatigen, konfigurierte Stabilisierung abwarten und zwei Bilder aufnehmen.
7. Alle Telemetrie-, Ereignis- und Steuerdaten kontinuierlich in einer Sessiondatei sichern.

4. Robustere Höhenlogik

4.1 Schwebelage

- Stabilkriterium: flying=true, takingOff=false, landing=false und $|verticalSpeed| \leq 0,25$ m/s.
- Nach 1,0 s stabilem Zustand wird nicht der letzte Einzelwert übernommen, sondern der Mittelwert der während dieses Fensters gemessenen relativen Höhen.
- Relative PStart-Höhe = aktuelle verticalDistance - verticalDistance beim PStart-Start. tofHeight bleibt Dokumentationswert.

4.2 Erreichen der Zielhöhe

- Ziel als erreicht, wenn der verbleibende Abstand $\leq 0,03$ m ist oder der Sollwert bereits überschritten wurde. PStart befiehlt niemals ein Absenken.
- Throttle-Staffelung: $>1,0$ m Rest = 260; $>0,35$ m Rest = 180; Nahbereich = 150. Der frühere Nahbereichswert 90 wurde angehoben, damit der +10-cm-Punkt nicht an einer möglichen Steuer-Totzone scheitert.
- Nach Zielerreichen wird Throttle auf Neutral gesetzt. Für 0,8 s muss $|verticalSpeed| \leq 0,35$ m/s sein.
- Fällt die Höhe mehr als 0,15 m unter das Ziel, wird der vertikale Aufstieg wieder aufgenommen. Nach 4 s darf die Stufe innerhalb dieses toleranten Höhenbands fortgesetzt werden.

Die Zahlenwerte sind bewusst konservative Erstwerte und müssen mit realen Flugtests gegen die tatsächliche Throttle-Kennlinie des Flugcontrollers validiert werden.

5. Horizontale Positionstoleranz

- Neue Einstellungen unter System -> PStart: Positions-Warnradius und Positions-Abbruchradius.
- Standardwerte: Warnradius 3,0 m; Abbruchradius 6,0 m.
- Innerhalb des Warnradius blockiert homeOffset keine Höhenstufe. Zwischen Warn- und Abbruchradius läuft PStart weiter, die Referenz erhält jedoch einen Positions-Warnstatus.
- Die Abbruchgrenze muss für 1,0 s kontinuierlich überschritten sein. Einzelne GPS-Sprünge lösen damit keinen sofortigen Abbruch aus.
- Auch mit dieser Toleranz gibt es in v1 keine automatische seitliche Positionskorrektur.

6. PStart-Sessionprotokoll

Jeder PStart-Lauf erzeugt eine eigene JSONL-Datei. Das Protokoll wird mit 5 Hz aufgezeichnet und enthaelt zusaetzlich explizite Ereignisdatensaetze. JSONL bedeutet: ein vollstaendiges JSON-Objekt pro Zeile, wodurch auch grosse Sessions einfach analysiert werden koennen.

6.1 Ablagestruktur

```
Pictures/PotensicProxy/Recognition/PStart_<SessionId>/
  PStart_<SessionId>_STEP0_step0_zero.jpg    <- Session-Master
  PStart_<SessionId>_STEP0_step0_max.jpg
  PStart_<SessionId>_REF_0_..._zero.jpg
  PStart_<SessionId>_REF_0_..._max.jpg
  ...
  PStart_<SessionId>_session.jsonl
```

6.2 Kontinuierliche Telemetrie

- Zeitstempel, Phase, stepId, referenceIndex und aktive Zielhoehe.
- verticalDistance, PStart-Relativhoehe, verticalSpeed und horizontalSpeed.
- GPS-Koordinaten, homeOffset, Satelliten, GPS-Status und - falls im Telemetriemodell vorhanden - gpsAccuracy.
- tofHeight als reine Zusatzinformation; Heading, Pitch, Roll und Gimbal-Pitch.
- Hardware-RC-Werte sowie die von PStart angeforderten Achsen. Damit ist insbesondere der automatische Throttle-Befehl nachtraeglich sichtbar.
- Flugstatus flying/takingOff/landing/returning und Positionsqualitaet.

6.3 Ereignisse

Unter anderem werden folgende Zustandswechsel explizit protokolliert: **SESSION_START, STEP0_READY, TAKEOFF_COMMAND, HOVER_DETECTED, REFERENCE_TARGETS_CALCULATED, ASCENT_STARTED, THROTTLE_COMMAND, TARGET_REACHED, TARGET_SETTLED, GIMBAL_PRESET, STABILIZATION_STARTED, IMAGE_CAPTURED, PSTART_COMPLETE, PSTART_ABORT** und **SESSION_END**.

7. Session-Masterbild

- Das erste STEP0-Bild ohne Zoom ist das Session-Masterbild. STEP0 bleibt weiterhin referenceEligible=false.
- Jedes Bild verweist im XMP auf SessionId, SessionMasterImage und SessionProtocolFile. Metadata-Schema wurde auf 2 und das PStart-Schema auf 1.1 angehoben.
- Nach Abschluss oder Abbruch wird das vollstaendige JSONL-Protokoll gzip-komprimiert und zusaetzlich in das Session-Master-JPEG eingebettet.
- Fuer die Einbettung werden eigene JPEG-APP15-Segmente mit der Signatur POTENSIC_PSTART_PROTOCOL verwendet. Das Protokoll wird bei Bedarf auf mehrere Segmente aufgeteilt und ist daher nicht auf die ca. 64 KiB eines einzelnen XMP-APP1-Segments begrenzt.
- Nach der Einbettung werden Dateigroesse und SHA-256 des Masterbildes neu ermittelt und die Media-Indexdaten aktualisiert. Normale JPEG-Decoder ignorieren APP15 und koennen das Bild weiterhin anzeigen.

8. Bildmetadaten

- Neu: stageld, heightError, qualityStatus, gpsQuality, positionQuality, Positions-Warn-/Abbruchgrenzen und PStart-Throttle-Befehl.
- Neu: sessionMaster, sessionMasterImage, sessionProtocolFile, sessionProtocolFormat und Hinweis auf die APP15-Einbettung.
- Weiterhin: vollstaendige Telemetrie, Kamera-, Gimbal-, Steuer- und Verbindungsdaten sowie EXIF-GPS und XMP-Payload.
- Qualitaetsstatus kann GOOD, WARNING_POSITION, WARNING_MOTION oder WARNING_POSITION_MOTION sein. Eine Warnung macht die Aufnahme nicht automatisch unbrauchbar.

9. Abbruchstatus

Das Session-Ende unterscheidet: **COMPLETED**, **ABORTED_MANUAL**, **ABORTED_GIMBAL**, **ABORTED_TIMEOUT**, **ABORTED_SAFETY** und **ABORTED_ERROR**. Der konkrete Abbruchgrund wird zusätzlich gespeichert.

10. Geaenderte Dateien

- .github/workflows/build-apk.yml
- app/build.gradle.kts
- app/src/main/assets/web/BUILD_INFO.txt
- app/src/main/assets/web/index.html
- app/src/main/assets/web/assets/modules/_package.js
- app/src/main/assets/web/assets/taf-pairing-runtime-v0.980.js (neu; v0.979 entfernt)
- VERSIONING.md
- webui/package.json
- webui/package-lock.json
- webui/index.html
- webui/public/assets/taf-pairing-runtime-v0.980.js (neu; v0.979 entfernt)
- webui/scripts/prepare-pairing-runtime.mjs (neu)
- webui/scripts/verify-embedded-version.mjs
- DOCUMENTATION/2026-10-11 - PotensicProxy_TAF - Projektdokumentation - PStart-CI-Runtime-Bereinigung - v0.980.docx/pdf

11. CI-Runtime-Bereinigung und Versionssynchronisierung

- Der hochgeladene GitHub-Actions-Log zeigt, dass npm ci und der komplette Vite-Build erfolgreich waren. Der Job brach erst im nachgelagerten Versionspruefer mit "Stale pairing runtime found: taf-pairing-runtime-v0.968.js" ab.
- Ursache: Die CI loescht vor dem Build zwar app/src/main/assets/web, Vite kopiert danach jedoch alle Dateien aus webui/public in das neue Embedded-Frontend. Wenn eine alte, noch getrackte Pairing-Runtime unter webui/public/assets liegt, wird sie deshalb erneut in app/src/main/assets/web/assets kopiert. emptyOutDir kann diese Quelle nicht bereinigen.
- v0.980 fuehrt deshalb webui/scripts/prepare-pairing-runtime.mjs ein. Das Skript liest die aktuelle Version aus webui/package.json, verlangt die dazu passende taf-pairing-runtime-Datei und entfernt vor dem Build jede anders versionierte Pairing-Runtime aus webui/public/assets.
- npm run build ruft diese Bereinigung jetzt explizit vor vue-tsc und Vite auf. Der GitHub-Actions-Workflow besitzt zusaetzlich einen eigenen Schritt "Remove stale pairing runtime sources" nach npm ci. Damit funktioniert die Bereinigung auch dann, wenn ein Aenderungspaket eine alte getrackte Datei nicht physisch loeschen konnte.
- Der Verifier prueft nun nicht nur die erzeugten Embedded-Assets, sondern auch webui/public/assets. Eine alte Runtime wird damit sowohl an der Quelle als auch im APK-WebUI erkannt. Erwarteter gemeinsamer Stand ist Frontend v0.980, Backend v0.980 und Pairing-Runtime v0.980.
- Die PStart-v0.978-Fluglogik, das 5-Hz-Sessionprotokoll, die Positionstoleranz und die Masterbild-Einbettung wurden durch diesen Buildfix nicht fachlich veraendert.

12. Validierung

Pruefung	Ergebnis	Hinweis
Stale-Runtime-Simulation	BESTANDEN	Eine kuenstlich angelegte taf-pairing-runtime-v0.968.js wurde vor dem Build automatisch entfernt; v0.980 blieb als einzige Runtime-Quelle erhalten.
Vue/TypeScript	BESTANDEN	node node_modules/vue-tsc/bin/vue-tsc.js --noEmit ohne Fehler.

Embedded-Versionspruefung	BESTANDEN	npm run verify:embedded: Frontend, Backend und Runtime sind v0.980; Source- und Embedded-Verzeichnis enthalten keine alte Runtime.
Embedded-JavaScript	BESTANDEN	77 eingebettete JavaScript-Dateien mit node --check ohne Syntaxfehler geprueft.
Vite-Bundle lokal	UMGEBUNG BLOCKIERT	Im mitgelieferten node_modules fehlt das Linux-Rollup-Optionalpaket. Der hochgeladene CI-Log belegt jedoch, dass der Vite-Build selbst erfolgreich war; der dortige Abbruch erfolgte erst beim alten Runtime-Verifier.
Android Gradle lokal	UMGEBUNG BLOCKIERT	Gradle-Wrapper benoetigt externen Download; im Arbeitsbereich steht kein Netzzugriff fuer den Wrapper zur Veruegung.
PStart-Funktionalitaet	UNVERAENDERT	Keine fachliche Aenderung an der PStart-v0.978-Fluglogik; v0.980 behebt die CI-/Runtime-Bereinigung.

13. Offene Hardware-Validierung

- Nach Installation zuerst im System-Tab pruefen: Frontend v0.980 und Backend v0.980. Bei Abweichung darf der Flugtest nicht als v0.980-Test gewertet werden.
- Nahbereichs-Throttle 150 und Hoeihenband im Freiflug pruefen; bei Bedarf nach Messdaten anpassen.
- Windtest mit erster Luftreferenz +0,10 m und mit Positionsabweichungen innerhalb Warnradius durchfuehren.
- Pruefen, dass die JSONL-Datei auf dem Ziel-Android im vorgesehenen PStart-Sessionordner sichtbar ist. Android Scoped Storage kann bei Nicht-Mediendateien geraetespezifisch relevant sein; das vollstaendige Protokoll bleibt unabhaengig davon im Session-Masterbild eingebettet.
- Nach Testflug JSONL gegen Live-System-Log und XMP der Bilder vergleichen.
- PLanding darf STEP0 weiterhin nicht als optische Referenz verwenden.

Diese Version fuehrt weiterhin keinen automatischen Pitch-/Roll-/Yaw-Regeleingriff waehrend des PStart-Aufstiegs aus. Die neue GPS-Toleranz ist eine Bewertungs- und Sicherheitslogik, keine horizontale Regelung.